

JWT HandsOn - Exercise

1. Add Dependencies in pom.xml

```
<!-- Spring Security -->

<dependency>

    <groupId>org.springframework.boot</groupId>

    <artifactId>spring-boot-starter-security</artifactId>

</dependency>

<!-- JWT -->

<dependency>

    <groupId>io.jsonwebtoken</groupId>

    <artifactId>jjwt</artifactId>

    <version>0.9.1</version>

</dependency>

<!-- Test -->

<dependency>

    <groupId>org.springframework.boot</groupId>

    <artifactId>spring-boot-starter-test</artifactId>

    <scope>test</scope>

</dependency>
```

Update Maven project after adding the dependencies.

2. Create Security Package

```
com.cognizant.springlearn.security
├── SecurityConfig.java
└── JwtAuthorizationFilter.java
```

3. SecurityConfig.java

```
package com.cognizant.springlearn.security;
```

```
import org.springframework.context.annotation.Bean;

import org.springframework.context.annotation.Configuration;

import
org.springframework.security.config.annotation.authentication.builders.AuthenticationMan
agerBuilder;

import org.springframework.security.config.annotation.web.builders.HttpSecurity;

import
org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;

import
org.springframework.security.config.annotation.web.configuration.WebSecurityConfigurerA
dapter;

import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;

import org.springframework.security.crypto.password.PasswordEncoder;
```

```
@Configuration
```

```
@EnableWebSecurity
```

```
public class SecurityConfig extends WebSecurityConfigurerAdapter {
```

```
    @Override
```

```
    protected void configure(AuthenticationManagerBuilder auth)
```

```
        throws Exception {
```

```
        auth.inMemoryAuthentication()
```

```
            .withUser("admin")
```

```
            .password(passwordEncoder().encode("pwd"))
```

```
            .roles("ADMIN")
```

```
            .and()
```

```
        .withUser("user")
        .password(passwordEncoder().encode("pwd"))
        .roles("USER");
    }
}
```

```
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
```

```
@Override
protected void configure(HttpSecurity http)
    throws Exception {
```

```
    http
        .csrf().disable()
        .httpBasic()
        .and()
        .authorizeRequests()

        .antMatchers("/authenticate")
        .hasAnyRole("USER", "ADMIN")

        .anyRequest()
        .authenticated()

        .and()
```

```
        .addFilter(  
            new JwtAuthorizationFilter(  
                authenticationManager());  
        )  
    }  
}
```

4. AuthenticationController.java

Create inside:

com.cognizant.springlearn.controller

```
package com.cognizant.springlearn.controller;
```

```
import java.util.Base64;
```

```
import java.util.Date;
```

```
import java.util.HashMap;
```

```
import java.util.Map;
```

```
import org.slf4j.Logger;
```

```
import org.slf4j.LoggerFactory;
```

```
import org.springframework.web.bind.annotation.GetMapping;
```

```
import org.springframework.web.bind.annotation.RequestHeader;
```

```
import org.springframework.web.bind.annotation.RestController;
```

```
import io.jsonwebtoken.JwtBuilder;
```

```
import io.jsonwebtoken.Jwts;
```

```
import io.jsonwebtoken.SignatureAlgorithm;
```

```
@RestController
```

```
public class AuthenticationController {
```

```
private static final Logger LOGGER =
    LoggerFactory.getLogger(
        AuthenticationController.class);

@GetMapping("/authenticate")
public Map<String, String> authenticate(
    @RequestHeader("Authorization")
    String authHeader) {

    LOGGER.info("START");
    LOGGER.debug(authHeader);

    String user = getUser(authHeader);

    String token = generateJwt(user);

    Map<String, String> map =
        new HashMap<>();

    map.put("token", token);

    LOGGER.info("END");

    return map;
}

private String getUser(String authHeader) {
```

```
String encodedCredentials =
    authHeader.substring(6);

byte[] decodedBytes =
    Base64.getDecoder()
        .decode(encodedCredentials);

String credentials =
    new String(decodedBytes);

LOGGER.debug(credentials);

return credentials.split(":")[0];
}

private String generateJwt(String user) {

    JwtBuilder builder =
        Jwts.builder();

    builder.setSubject(user);

    builder.setIssuedAt(
        new Date());

    builder.setExpiration(
        new Date(
```

```
        System.currentTimeMillis()
        + 1200000));
```

```
        builder.signWith(
            SignatureAlgorithm.HS256,
            "secretkey");

        return builder.compact();
    }
}
```

5. JwtAuthorizationFilter.java

Create inside:

com.cognizant.springlearn.security

package com.cognizant.springlearn.security;

import java.io.IOException;

import java.util.ArrayList;

import javax.servlet.FilterChain;

import javax.servlet.ServletException;

import javax.servlet.http.HttpServletRequest;

import javax.servlet.http.HttpServletResponse;

import org.slf4j.Logger;

import org.slf4j.LoggerFactory;

import org.springframework.security.authentication.AuthenticationManager;

```
import
org.springframework.security.authentication.UsernamePasswordAuthenticationToken;

import org.springframework.security.core.context.SecurityContextHolder;

import org.springframework.security.web.authentication.www.BasicAuthenticationFilter;

import io.jsonwebtoken.Claims;
import io.jsonwebtoken.Jws;
import io.jsonwebtoken.JwtException;
import io.jsonwebtoken.Jwts;

public class JwtAuthorizationFilter
    extends BasicAuthenticationFilter {

    private static final Logger LOGGER =
        LoggerFactory.getLogger(
            JwtAuthorizationFilter.class);

    public JwtAuthorizationFilter(
        AuthenticationManager authenticationManager) {

        super(authenticationManager);

        LOGGER.info("START");
    }

    @Override
```



```
protected void doFilterInternal(  
    HttpServletRequest req,  
    HttpServletResponse res,  
    FilterChain chain)  
  
    throws IOException,  
    ServletException {  
  
    LOGGER.info("START");  
  
    String header =  
        req.getHeader("Authorization");  
  
    LOGGER.debug(header);  
  
    if (header == null ||  
        !header.startsWith("Bearer ")) {  
  
        chain.doFilter(req, res);  
        return;  
    }  
  
    UsernamePasswordAuthenticationToken  
        authentication =  
        getAuthentication(req);  
  
    SecurityContextHolder  
        .getContext()
```

```
        .setAuthentication(authentication);

        chain.doFilter(req, res);

        LOGGER.info("END");
    }
}
```

```
private
UsernamePasswordAuthenticationToken
getAuthentication(
    HttpServletRequest request) {

    String token =
        request.getHeader(
            "Authorization");

    try {

        Jws<Claims> jws =
            Jwts.parser()

                .setSigningKey(
                    "secretkey")

                .parseClaimsJws(
                    token.replace(
                        "Bearer ",
                        ""));
    }
```

```

String user =
    jws.getBody()
        .getSubject();

LOGGER.debug(user);

if (user != null) {

    return new
        UsernamePasswordAuthenticationToken(
            user,
            null,
            new ArrayList<>());
    }

} catch (JwtException ex) {

    return null;
    }

return null;
}
}

```

6. application.properties

```

server.port=8090
logging.level.com.cognizant=DEBUG

```

7. Run the Application

Run:

```
mvn clean install
```

Then run:

```
SpringLearnApplication.java
```

8. Test Basic Authentication

Correct User

```
curl -s -u user:pwd http://localhost:8090/authenticate
```

Output:

```
{
  "token":"eyJhbGc..."
}
```

Wrong Password

```
curl -s -u user:pwd1 http://localhost:8090/authenticate
```

Output:

```
{
  "status":401,
  "error":"Unauthorized"
}
```

9. Test JWT

Get token:

```
curl -s -u user:pwd http://localhost:8090/authenticate
```

Copy the token.

Call service:

```
curl -s \
-H "Authorization: Bearer YOUR_TOKEN" \
http://localhost:8090/countries
```

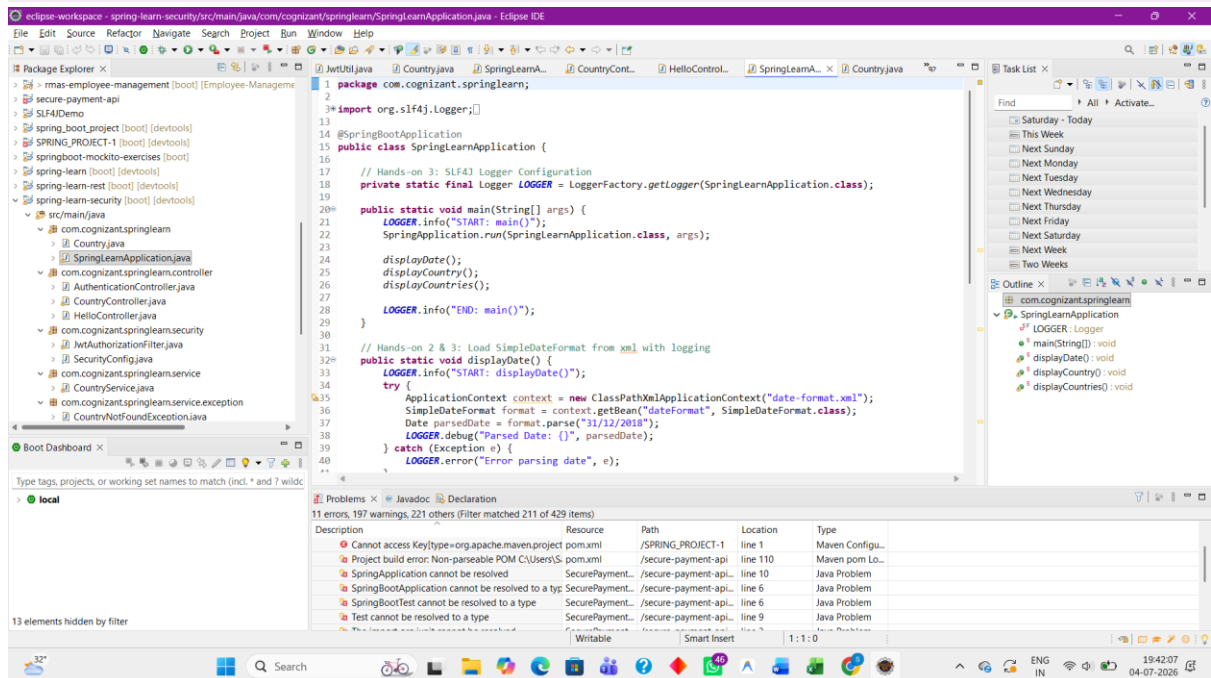
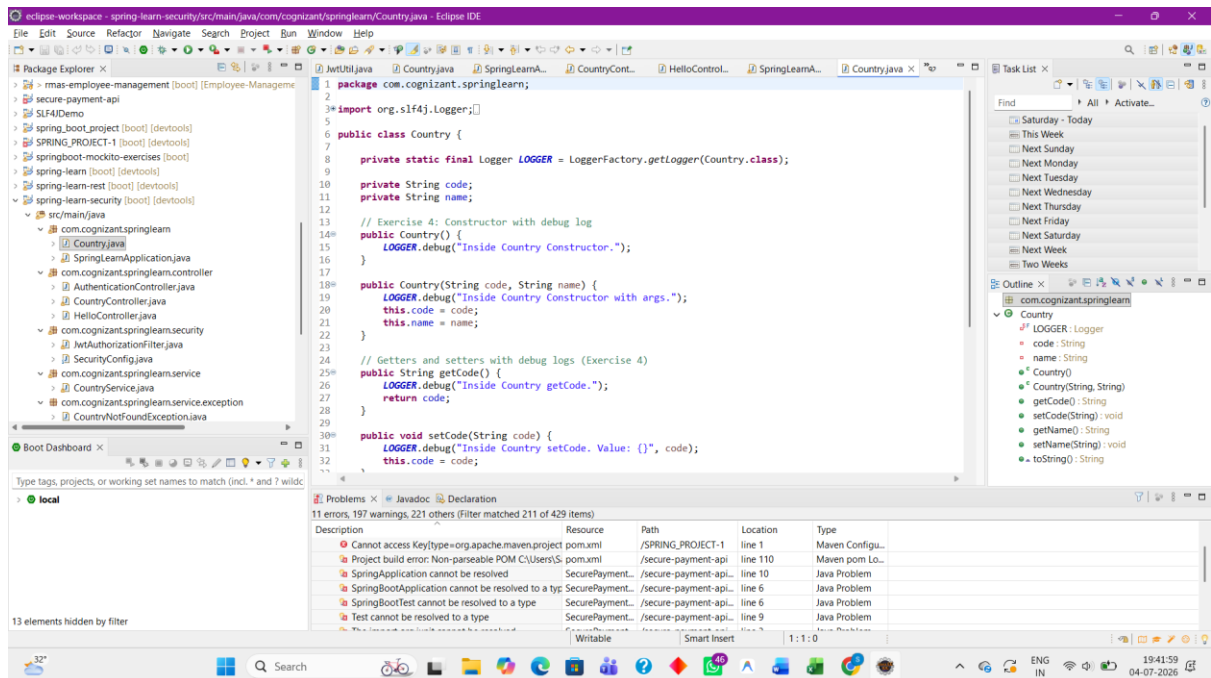
Output:

```
[
  {
    "code": "IN",
    "name": "India"
  },
  {
    "code": "US",
    "name": "United States"
  },
  {
    "code": "JP",
    "name": "Japan"
  },
  {
    "code": "DE",
    "name": "Germany"
  }
]
```

```

  v  spring-learn-security [boot] [devtools]
    v  src/main/java
      v  com.cognizant.springlearn
        >  Country.java
        >  SpringLearnApplication.java
      v  com.cognizant.springlearn.controller
        >  AuthenticationController.java
        >  CountryController.java
        >  HelloController.java
      v  com.cognizant.springlearn.security
        >  JwtAuthorizationFilter.java
        >  SecurityConfig.java
      v  com.cognizant.springlearn.service
        >  CountryService.java
      v  com.cognizant.springlearn.service.exception
        >  CountryNotFoundException.java
    >  src/main/resources
    >  src/test/java
    >  JRE System Library [JavaSE-21]
    >  Maven Dependencies
    >  target/generated-sources/annotations
    >  target/generated-test-sources/test-annotations
    >  src
    >  target
    >  mvnw
  ]

```



Eclipse IDE workspace: spring-learn-security/src/main/java/com/cognizant/springlearn/controller/AuthenticationController.java

Package Explorer: rmas-employee-management [boot] [Employee-Management], secure-payment-api, SLF4JDemo, spring_boot_project [boot] [devtools], SPRING_PROJECT-1 [boot] [devtools], springboot-mockito-exercises [boot], spring-learn [boot] [devtools], spring-learn-test [boot] [devtools], spring-learn-security [boot] [devtools], src/main/java, com.cognizant.springlearn, Country.java, SpringLearnApplication.java, com.cognizant.springlearn.controller, AuthenticationController.java, CountryController.java, HelloController.java, com.cognizant.springlearn.security, JwtAuthorizationFilter.java, SecurityConfig.java, com.cognizant.springlearn.service, CountryService.java, com.cognizant.springlearn.service.exception, CountryNotFoundException.java

Boot Dashboard: local

Type tags, projects, or working set names to match (incl. * and ? wildcards)

13 elements hidden by filter

```
1 package com.cognizant.springlearn.controller;
2
3 import java.util.Base64;
4
5 @RestController
6 public class AuthenticationController {
7
8     private static final Logger LOGGER = LoggerFactory.getLogger(AuthenticationController.class);
9
10    @GetMapping("/authenticate")
11    public Map<String, String> authenticate(@RequestHeader("Authorization") String authHeader) {
12        LOGGER.info("START: authenticate()");
13        LOGGER.debug("Authorization Header: {}", authHeader);
14
15        String user = getUser(authHeader);
16        LOGGER.debug("Decoded User: {}", user);
17
18        String token = generateJwt(user);
19        LOGGER.debug("Generated JWT: {}", token);
20
21        Map<String, String> response = new HashMap<>();
22        response.put("token", token);
23
24        LOGGER.info("END: authenticate()");
25        return response;
26    }
27
28    private String getUser(String authHeader) {
29        LOGGER.info("START: getUser()");
30        LOGGER.debug("Header: {}", authHeader);
31    }
32}
```

Problems: 11 errors, 197 warnings, 221 others (Filter matched 211 of 429 items)

Description	Resource	Path	Location	Type
Cannot access Key<type=org.apache.maven.project.pom.xml>	pom.xml	/SPRING_PROJECT-1	line 1	Maven Configur...
Project build error: Non-parseable POM C:\Users\S\pom.xml	pom.xml	/secure-payment-api	line 110	Maven pom Lo...
SpringApplication cannot be resolved	SecurePayment...	/secure-payment-api	line 10	Java Problem
SpringBootApplication cannot be resolved to a type	SecurePayment...	/secure-payment-api	line 6	Java Problem
SpringBootTest cannot be resolved to a type	SecurePayment...	/secure-payment-api	line 6	Java Problem
Test cannot be resolved to a type	SecurePayment...	/secure-payment-api	line 9	Java Problem

Outline: com.cognizant.springlearn.controller, AuthenticationController, LOGGER: Logger, authenticate(String): Map<String, String>, getUser(String): String, generateJwt(String): String

Eclipse IDE workspace: spring-learn-security/src/main/java/com/cognizant/springlearn/controller/CountryController.java

Package Explorer: rmas-employee-management [boot] [Employee-Management], secure-payment-api, SLF4JDemo, spring_boot_project [boot] [devtools], SPRING_PROJECT-1 [boot] [devtools], springboot-mockito-exercises [boot], spring-learn [boot] [devtools], spring-learn-test [boot] [devtools], spring-learn-security [boot] [devtools], src/main/java, com.cognizant.springlearn, Country.java, SpringLearnApplication.java, com.cognizant.springlearn.controller, AuthenticationController.java, CountryController.java, HelloController.java, com.cognizant.springlearn.security, JwtAuthorizationFilter.java, SecurityConfig.java, com.cognizant.springlearn.service, CountryService.java, com.cognizant.springlearn.service.exception, CountryNotFoundException.java

Boot Dashboard: local

Type tags, projects, or working set names to match (incl. * and ? wildcards)

13 elements hidden by filter

```
1 package com.cognizant.springlearn.controller;
2
3 import java.util.ArrayList;
4
5 @RestController
6 public class CountryController {
7
8     private static final Logger LOGGER = LoggerFactory.getLogger(CountryController.class);
9
10    @Autowired
11    private CountryService countryService;
12
13    public CountryController() {
14        LOGGER.debug("Inside CountryController Constructor.");
15    }
16
17    @RequestMapping(value = "/country", method = RequestMethod.GET)
18    public Country getCountryIndia() {
19        LOGGER.info("START: getCountryIndia()");
20        ApplicationContext context = new ClassPathXmlApplicationContext("country.xml");
21        Country country = context.getBean("in", Country.class);
22        LOGGER.info("END: getCountryIndia()");
23        return country;
24    }
25
26    @SuppressWarnings("unchecked")
27    @GetMapping("/countries")
28    public List<Country> getAllCountries() {
29        LOGGER.info("START: getAllCountries()");
30        ApplicationContext context = new ClassPathXmlApplicationContext("country.xml");
31        List<Country> countries = context.getBean("countryList", ArrayList.class);
32    }
33}
```

Problems: 11 errors, 197 warnings, 221 others (Filter matched 211 of 429 items)

Description	Resource	Path	Location	Type
Cannot access Key<type=org.apache.maven.project.pom.xml>	pom.xml	/SPRING_PROJECT-1	line 1	Maven Configur...
Project build error: Non-parseable POM C:\Users\S\pom.xml	pom.xml	/secure-payment-api	line 110	Maven pom Lo...
SpringApplication cannot be resolved	SecurePayment...	/secure-payment-api	line 10	Java Problem
SpringBootApplication cannot be resolved to a type	SecurePayment...	/secure-payment-api	line 6	Java Problem
SpringBootTest cannot be resolved to a type	SecurePayment...	/secure-payment-api	line 6	Java Problem
Test cannot be resolved to a type	SecurePayment...	/secure-payment-api	line 9	Java Problem

Outline: com.cognizant.springlearn.controller, CountryController, LOGGER: Logger, countryService: CountryService, CountryController(), getCountryIndia(): Country, getAllCountries(): List<Country>, getCountry(String): Country

